Course Registration System

Stack Used:

1. Python
2. Oracle21c

Tool:

1. VS Code
2. Oracle21c Database

Project Made By: **Arman Bin Alauddin**

Contents

Abstract 2
Acknowledgment 2
1. Introduction 3
2. Database Design Concepts Used 3
3. Objectives and Scope 3
4. Methodology 4
5. ER Diagram, Crucial Code Snippet and Explanation 5
Snippet 1: Block Re-registration of Completed Courses 6
Snippet 2: Course Listing with Automatic Filtering 6
Snippet 3: Drop Course 7
Example Output 8
6. Innovation & Uniqueness 9
7. Results & Discussion 9
Results: 9
Discussion: 9
8. Summary 10
9. References 10

Abstract

The Course Registration System is a console-based application developed in Python, integrated with Oracle 21c Database, designed for university environments to manage student course registrations efficiently. The system authenticates students using their Student ID and password, displays available courses filtered by faculty and credit hours, and processes enrollments securely.

Key features include:

1. Student login with ID and password authentication.
2. Course browsing with automatic filtering of completed and already-enrolled courses.
3. Course registration with duplicate and repeat-completion blocking.
4. Course dropping with audit trail preservation.
5. Admin panel for managing students and marking course completions.
6. Permanent data storage using Oracle 21c relational database with five tables.

This project demonstrates the integration of Python programming, Oracle SQL, relational database design, and modular software architecture while simulating real-world university registration operations.

Acknowledgment

First and foremost, I express my gratitude to Almighty Allah for granting me the ability to complete this project.

I am also grateful to my Pie.

1. Introduction

This project is a Course Registration System developed in Python with Oracle 21c Database that allows a university to efficiently manage student course registrations each semester. It also includes a course completion tracking feature with a registration-block mechanism for already-completed courses. The system collects student credentials including Student ID and password, then presents available courses with details such as course name, credit hours, and faculty name. It supports multiple enrollment operations, applies business rules such as duplicate registration prevention, and stores all records permanently in Oracle. The generated enrollments and student history are stored in structured relational tables for long-term record-keeping. The project provides a simple console interface suitable for university registration counters and student self-service terminals.

2. Database Design Concepts Used

1. Tables & Relationships: Five relational tables (STUDENT, FACULTY, COURSE, ENROLLMENT, COMPLETED_COURSE) are connected through primary and foreign keys to enforce referential integrity.

2. Modularity: Separate Python modules handle authentication (auth.py), course viewing (courses.py), enrollment operations (enrollment.py), admin functions (admin.py), and database connection (db.py).

3. Reusability: The get_connection() function in db.py is imported and reused across all modules, eliminating duplicate connection code.

4. SQL Constraints: Oracle CHECK, UNIQUE, NOT NULL, and REFERENCES constraints enforce data integrity at the database level, independent of application code.

5. Sequences: Oracle sequences (seq_student, seq_course, etc.) auto-generate unique primary key values, equivalent to AUTO_INCREMENT in other databases.

6. Bind Variables: All SQL queries use Oracle bind variables (:sid, :pwd) to prevent SQL injection attacks and improve query performance.

3. Objectives and Scope

This Course Registration System can be used in:

1. Universities and colleges: For managing semester-wise course registrations with faculty and credit hour information.

2. Student self-service portals: To allow students to register and drop courses independently through a secure login.

3. Academic administration: Administrators can add new students, view all registered students, and mark course completions that permanently block re-registration.

4. Educational purposes: It serves as an excellent demonstration of relational database design, SQL constraints, Python-Oracle integration, and modular software architecture.

4. Methodology

1. Requirement Analysis: Identified essential registration tasks — student login, course browsing, enrollment, dropping, and completion tracking.

2. Database Design: Five relational tables designed with appropriate primary keys, foreign keys, and CHECK constraints to enforce all business rules at the database level.

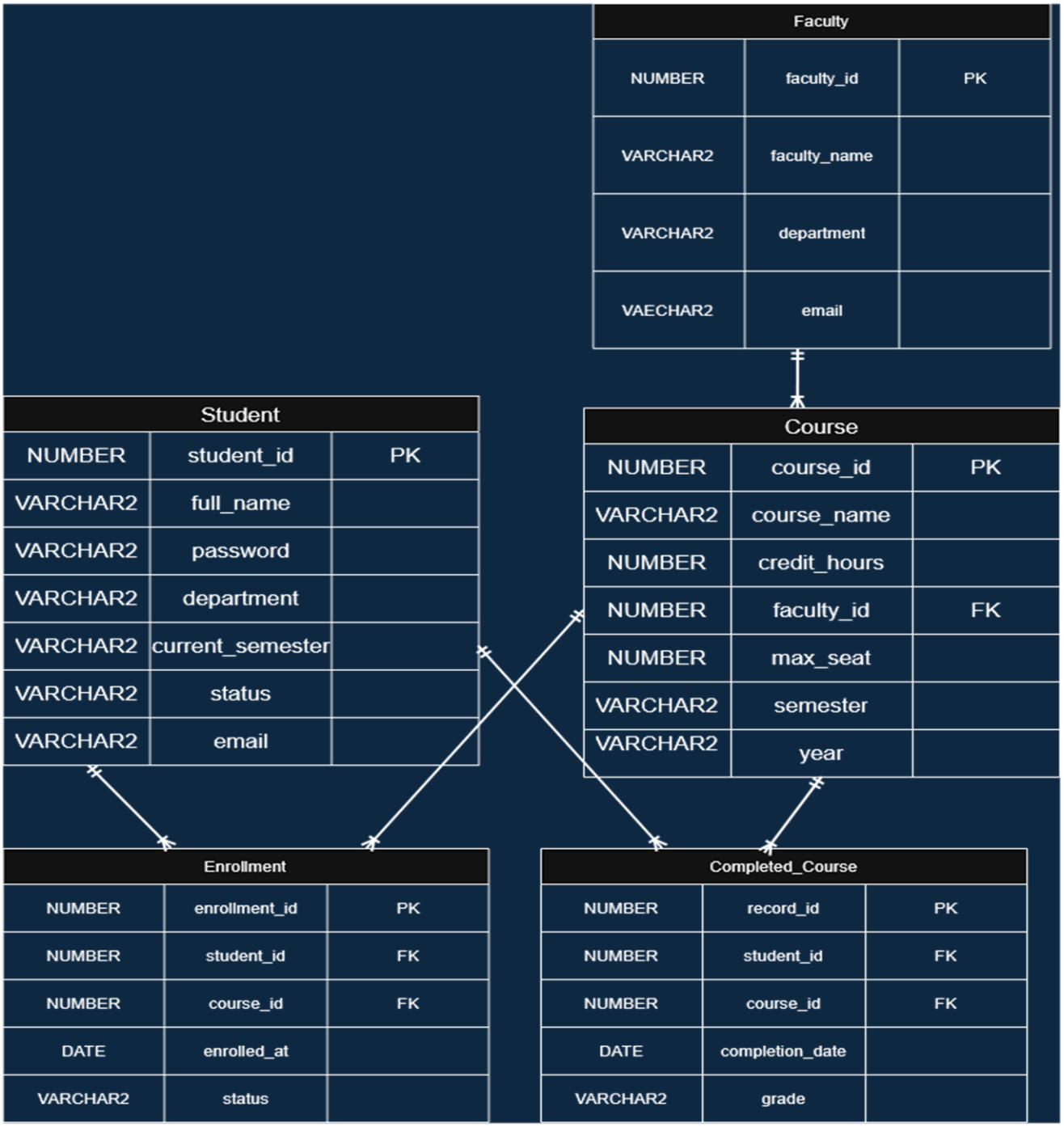
3. Modular Python Architecture: Separate files for each concern — db.py for connections, auth.py for login, courses.py for listings, enrollment.py for transactions, admin.py for management, and main.py for navigation.

4. Oracle Integration: Used the oracledb Python driver to connect to Oracle 21c. All INSERT and UPDATE operations are followed by conn.commit() for permanent storage.

5. Error Validation: Input validation ensures only valid integers are accepted for IDs and menu choices. Oracle IntegrityError exceptions are caught and handled with user-friendly messages.

6. Testing: Verified functionalities including student login, course registration, duplicate blocking, completed-course blocking, course dropping, and admin operations.

5. ER Diagram, Crucial Code Snippet and Explanation



Snippet 1: Block Re-registration of Completed Courses

```
def register_course(student_id: int):
    # Check if already completed
    cursor.execute("""
        SELECT COUNT(*)
        FROM   COMPLETED_COURSE
        WHERE  student_id = :sid
        AND    course_id  = :cid
    """, sid=student_id, cid=course_id)

    already_done = cursor.fetchone()[0]

    if already_done > 0:
        print('  BLOCKED: You already completed this course.')
        print('  Re-registration of completed courses is not allowed.')
        return

    # Proceed with INSERT if not completed
    cursor.execute("""
        INSERT INTO ENROLLMENT
            (enrollment_id, student_id, course_id, enrolled_at, status)
        VALUES
            (seq_enrollment.NEXTVAL, :sid, :cid, SYSDATE, 'active')
    """, sid=student_id, cid=course_id)
    conn.commit()
```

Explanation:

1. The function first queries the COMPLETED_COURSE table to check if the student has already finished this course.

2. COUNT(*) returns 0 if the course was never completed, or 1 if it was.

3. If already completed, a block message is printed and the function returns immediately — no INSERT is executed.

4. If the check passes, the enrollment is inserted with seq_enrollment.NEXTVAL for a unique ID and SYSDATE for the current timestamp.

5. conn.commit() permanently saves the record to Oracle — without this, the data would be lost when the connection closes.

Snippet 2: Course Listing with Automatic Filtering

```
cursor.execute("""
    SELECT c.course_id, c.course_name, c.credit_hours,
           f.faculty_name, c.semester, c.year_val
```

```
    FROM   COURSE  c
    JOIN   FACULTY f ON c.faculty_id = f.faculty_id
    WHERE  c.course_id NOT IN (
               SELECT course_id FROM COMPLETED_COURSE
               WHERE  student_id = :sid
           )
    AND    c.course_id NOT IN (
               SELECT course_id FROM ENROLLMENT
               WHERE  student_id = :sid
               AND    status = 'active'
           )
    ORDER BY c.course_id
""", sid=student_id)
```

Explanation:

1. The query retrieves all courses and joins with the FACULTY table to include the faculty name in one SQL call.

2. The first NOT IN subquery hides all courses the student has already completed — this is the core 'block repeats' feature visible to the student.

3. The second NOT IN subquery hides courses the student is currently enrolled in, so they are not shown as available again.

4. Both subqueries use the same :sid bind variable, which Oracle fills in safely without risk of SQL injection.

5. ORDER BY ensures courses always appear in a consistent, predictable order by ID.

Snippet 3: Drop Course

```
cursor.execute("""
    DELETE FROM ENROLLMENT
    WHERE enrollment_id = :eid AND student_id = :sid
""", eid=enroll_id, sid=student_id)

affected = cursor.rowcount
conn.commit()

if affected == 1:
    print('  Course dropped successfully.')
else:
    print('  Enrollment not found or already dropped.')
```

Explanation:

1. The WHERE clause requires both the enrollment ID and the student ID to match, preventing one student from accidentally dropping another student's enrollment.

2. cursor.rowcount returns the number of rows actually updated — if 0, the enrollment was not found or did not belong to this student.

3. conn.commit() permanently saves the status change to Oracle.

Example Output

```
============================================================
  COURSE REGISTRATION SYSTEM
============================================================
  Connecting to Oracle...
  Oracle connected successfully.

============================================================
  MAIN MENU
============================================================
  1. Student Login
  2. Admin Panel
  0. Exit

  Choice: 1

────────────────────────────────────────────────────────
  STUDENT LOGIN
────────────────────────────────────────────────────────
  Student ID : 41240202189
  Password   : 12345
  Login successful! Welcome, Arman.

  AVAILABLE COURSES
────────────────────────────────────────────────────────
  ID    Course Name                  Credits  Faculty
  ──────────────────────────────────────────────────
  2     Database Management          3        Dr. Islam
  3     Digital Electronics          3        Dr. Hossain
  4     Object Oriented Prog         3        Dr. Rahman
  5     Computer Networks            3        Dr. Islam

  [Completed courses are automatically hidden]
```

```
Enter Course ID to register (0 to cancel): 2
Successfully registered for "Database Management"!
```

6. Innovation & Uniqueness

In a basic course registration system, data is often lost after the program ends and there is no mechanism to prevent re-registration of completed courses. Our project includes all necessary features that enhance the reliability and realism of a university registration process. The uniqueness of this project lies in its practical academic features built on top of a standard registration flow:

1. Completion-Based Blocking: Completed courses are permanently blocked from re-registration using a dedicated COMPLETED_COURSE table queried before every enrollment attempt.

2. Modular Python Architecture: Each concern is separated into its own file — authentication, course viewing, enrollment, and admin — making the codebase maintainable and extensible.

3. Oracle Constraint Enforcement: Business rules are enforced at two levels — in Python code and in Oracle CHECK/UNIQUE constraints — ensuring data integrity even if the application code is bypassed.

4. Persistent Data: All student data, enrollments, and completion records are stored permanently in Oracle 21c, surviving across multiple sessions and program restarts.

7. Results & Discussion

Results:

1. Students successfully authenticate with their Student ID and password — invalid credentials are rejected with a clear error message.

2. The course listing dynamically filters out completed and currently-enrolled courses, showing only genuinely available options.

3. Re-registration of completed courses is blocked at the Python level (COUNT check) and reinforced by Oracle constraints.

4. All enrollment and drop operations are committed to Oracle and remain available across sessions.

5. The admin panel successfully adds new students and marks courses as completed, immediately affecting what the student sees in their course list.

Discussion:

The system demonstrates how relational database design and Python programming can simulate a real-world university registration process. Compared to a basic registration system that only holds data in memory, the inclusion of Oracle persistent storage, a completion-blocking mechanism, and a full audit trail makes this program closer to a practical academic solution. The separation of

concerns across multiple Python modules also makes it straightforward to extend — for example, adding a GUI, a web interface, or additional features like waitlisting or prerequisite checking.

8. Summary

In this project, a console-based Course Registration System was created using Python and Oracle 21c Database. It enables students to log in securely, browse available courses, register for and drop courses, and view their enrollment history and completed courses. The system is backed by five relational tables in Oracle with proper constraints, sequences, and foreign keys. The design follows modular software principles — each Python file handles one concern — making the system structured and maintainable. While console-based, this system reflects how real-world university registration processes work and lays a solid foundation for more advanced systems involving web interfaces, notification systems, or full student information portals.

9. References

1. Oracle Python Driver (oracledb) — https://python-oracledb.readthedocs.io/en/latest/

2. W3Schools SQL Tutorial — https://www.w3schools.com/sql/

3. Oracle 21c Documentation — https://docs.oracle.com/en/database/oracle/oracle-database/21/

4. Python Official Documentation — https://docs.python.org/3/

5. Oracle PL/SQL Sequences — https://docs.oracle.com/cd/B19306_01/server.102/b14200/statements_6015.htm